

# JAVA INTERVIEW PREPARATION

CHAPTER 30

MEMORY LEAKS AND OUTFOFMEMORYERROR

Senior / Lead Java Developer Textbook Edition

Full reading material - not summarized

# Memory Leaks and OutOfMemoryError: Retention Paths, Heap Dumps, and Native Failure

Senior / Lead Java Developer Reading Chapter

Java prevents many forms of manual-memory corruption, but it cannot know when the application no longer needs a reachable object. A collection, listener, ThreadLocal, class loader, or queue can retain valid objects forever. OutOfMemoryError is also not synonymous with a heap leak: Metaspace, direct memory, native threads, oversized arrays, and operating-system limits produce different failures.

## What a Java Memory Leak Means

A Java heap leak is unwanted reachability.

```
GC root -> singleton -> map -> entry -> object graph no longer needed
```

The collector is correct to keep the graph because a strong path exists. Diagnosis asks why the root path remains and which ownership or eviction rule is missing.

Normal heap usage often forms a sawtooth as allocation grows and collection reduces used memory. A stronger leak signal is a post-collection live set that rises across comparable workload periods.

```
healthy: /\ /\ /\ similar floor after collections
leaking: /\ /\ /\ floor trends upward
```

Traffic growth, cache warm-up, batch phase, and legitimate state growth can produce the same shape. Compare equivalent operating conditions.

## Common Retention Patterns

### Unbounded Maps and Caches

```
private final Map<String, Result> cache = new ConcurrentHashMap<>();

Result load(String key) {
    return cache.computeIfAbsent(key, repository::load);
}
```

Thread safety does not provide eviction. Define maximum size or weight, expiry, refresh, key cardinality, and behavior when the bound is reached. A mature cache library usually handles concurrency and eviction better than a custom map.

### Listeners and Registrations

A long-lived publisher retains every registered listener until removal. Return a registration handle and close it.

```
try (Registration registration = bus.register(listener)) {
    runComponent();
}
```

Weak listeners change semantics and can disappear unexpectedly; explicit lifecycle is usually clearer.

## ThreadLocal Values

Pool threads outlive requests. A ThreadLocal value can retain request data, class loaders, buffers, and security context.

```
try {
    context.set(requestContext);
    process();
} finally {
    context.remove();
}
```

ThreadLocalMap keys are weakly referenced, but values are not automatically harmless when a key is collected. Stale entries may remain until table activity cleans them. Always remove values you install on long-lived threads.

## Queues and Backlogs

An unbounded executor or message queue retains tasks and their captured object graphs when producers outrun consumers. This is both a memory and latency leak.

arrival rate > completion rate -> queue depth and oldest age grow continuously

Bound queues, apply backpressure or rejection, and monitor queue age. Increasing heap merely stores a larger failure.

## Static Fields and Class Loaders

Static lifetime follows the defining class loader, not necessarily the entire process. A parent-loaded static registry holding a child-loaded object can prevent an application loader and all its classes from unloading. Redeployment leaks often appear as growing loader count and Metaspace.

## Native and Direct Memory

Direct buffers, JNI libraries, compression engines, TLS libraries, and memory mappings may grow while heap remains stable. Heap dumps do not contain the native payload. Use Native Memory Tracking where applicable, direct-buffer metrics, OS maps, and native tooling.

## OutOfMemoryError Is a Family of Diagnoses

Always record the full detail message.

```
Java heap space
GC overhead limit exceeded
Metaspace
Compressed class space
Direct buffer memory
unable to create native thread
Requested array size exceeds VM limit
native allocation failure or out-of-swap indications
```

**Java heap space** means an allocation could not be satisfied within available heap after collection and growth attempts. It may be a leak, an undersized heap, an allocation burst, or an object larger than remaining contiguous collector capacity.

**GC overhead limit exceeded** indicates the JVM spent excessive effort collecting while recovering very little, under the relevant HotSpot policy. Disabling the limit does not fix the retention or capacity problem.

**Metaspace** concerns class metadata. **unable to create native thread** concerns native memory or OS process/thread limits. **Requested array size exceeds VM limit** indicates an implementation-size limit, not simply insufficient configured heap.

## Container OOM Kill Versus Java OOME

An operating system or container can terminate the process when total memory exceeds its limit. The JVM may not have an opportunity to throw `OutOfMemoryError` or write a heap dump.

```
container limit
= heap + metaspace + code cache + stacks + direct/native + JVM + mapped pages
```

Check container termination reason, cgroup metrics, kernel logs where available, peak RSS, and configured heap ergonomics. Leave native headroom rather than assigning the full container limit to `-Xmx`.

## Capturing Evidence Before It Disappears

Useful startup options can include:

```
-XX:+HeapDumpOnOutOfMemoryError
-XX:HeapDumpPath=/diagnostics
-XX:ErrorFile=/diagnostics/hs_err_pid%p.log
-Xlog:gc*:file=/diagnostics/gc.log:time,uptime,level,tags
```

Ensure the directory exists, is writable, has enough space, and persists beyond container restart. A heap dump can approach heap size and may contain credentials, personal data, and application payloads. Protect it as sensitive production data.

Operational policies sometimes use `-XX:+ExitOnOutOfMemoryError` or `-XX:+CrashOnOutOfMemoryError`. These materially change availability and evidence behavior and should be chosen with platform restart and dump policy, not copied blindly.

## Heap Dumps

A heap dump captures heap objects, classes, and reference relationships at a point in time.

```
jcmd <pid> GC.heap_dump filename=/diagnostics/heap.hprof
```

Creating a dump can pause the process, consume CPU and disk bandwidth, and destabilize an already unhealthy service. Confirm target process, free disk, security, and operational impact.

The dump answers what the heap retained, not why the application chose to retain it. Analysis combines object graphs with domain knowledge and time-series evidence.

## Shallow Size, Retained Size, and Dominators

Shallow size is the memory occupied by one object itself. Retained size estimates memory that would become reclaimable if that object were removed, subject to analyzer semantics. A dominator is an object through which all paths to another object pass.

```
Root -> Cache -> Map -> 2 million entries
      ^ dominator with large retained set
```

A small `HashMap` object can dominate gigabytes of keys and values. Sorting only by shallow size can miss the ownership problem. Dominator trees and retained-size views identify major retaining structures.

## Paths to GC Roots

After identifying suspicious instances, trace paths to GC roots.

```
large byte[]  
  <- Response.body  
  <- FutureTask.callable  
  <- executor work queue  
  <- static ExecutorHolder.pool  
  <- system class loader root
```

This path reveals an executor backlog rather than a byte-array leak. Exclude weak or soft paths carefully according to the question being asked. Multiple roots may retain the same object.

## Histograms and Comparative Analysis

Class histograms give counts and shallow bytes by class.

```
jcmd <pid> GC.class_histogram
```

Histograms are lower-cost triage than full heap dumps but do not provide complete ownership paths. Compare counts over time. A million String objects may be business data, duplicate parsing, cache keys, or queued payloads; class name alone does not prove cause.

Two heap dumps captured at comparable workload points can expose growing classes and retaining owners. Differences between incomparable traffic phases can mislead.

## Native Memory Diagnosis

Enable HotSpot NMT at startup when native accounting is required.

```
-XX:NativeMemoryTracking=summary  
jcmd <pid> VM.native_memory baseline  
jcmd <pid> VM.native_memory summary.diff
```

NMT covers HotSpot-tracked categories, not all third-party native allocations. If NMT is stable but RSS grows, investigate JNI libraries, allocator fragmentation, mapped files, direct-buffer pools, and OS-level memory maps.

For native-thread exhaustion, inspect platform thread count, [-Xss](#), process limits, and thread creation sites. A thread dump and JFR thread events may reveal per-request thread creation or executors never shut down.

## Memory Leak Investigation Workflow

1. Preserve the exact error, timestamps, deployment, traffic, and container state.
2. Determine heap, Metaspace, direct, thread, or broader native failure.
3. Examine GC logs and post-GC live-set trend.
4. Capture a histogram, JFR, heap dump, or NMT evidence appropriate to the area.
5. In heap analysis, find dominant retained sets and paths to roots.
6. Connect the retaining owner to an absent lifecycle, bound, or eviction rule.
7. Reproduce with a smaller controlled workload and verify the fix by trend.

Increasing memory may be a valid capacity adjustment when the live set is legitimate. It is not a complete response to unbounded retention.

## Leak, Capacity Shortage, and Allocation Surge

The same `Java heap space` message can emerge from different dynamics:

leak: post-GC live set rises over comparable intervals  
capacity shortage: post-GC live set is stable but legitimately near maximum  
allocation surge: live set may recover, but a burst outruns collection/headroom  
oversized object: one array or graph cannot be accommodated safely

A leak calls for an ownership correction. A legitimate live set may require more memory, smaller retained business state, or partitioning. A burst may require admission control, streaming, chunking, or additional headroom. Treating all four as "increase Xmx" can hide an unbounded defect; treating all four as "find a leak" can waste time when the capacity model is simply wrong.

Fragmentation and collector-specific allocation rules can contribute even when total free bytes appear sufficient. Large or humongous allocations may need suitable regions or contiguous logical capacity. The full GC log around the failure explains far more than a dashboard's last heap percentage.

## Cache Leaks Are Often Cardinality Leaks

A cache must define the population it is willing to retain. Entry count alone is insufficient when values vary from one kilobyte to 50 megabytes.

key cardinality: tenant x user x locale x query x version  
weight: serialized bytes or estimated retained cost  
lifecycle: expire-after-write/access, refresh, explicit invalidation  
failure policy: what happens when loader fails or cache is full

Unexpected key cardinality is a frequent cause: request IDs, timestamps, unconstrained search strings, or versioned identifiers make every lookup unique. A bounded cache can still hurt if its bound is much larger than the memory budget or if eviction churn causes high allocation and latency.

Loading caches require failure semantics. A future or placeholder for an in-progress load can retain request arguments, exceptions, or waiters indefinitely if completion is lost. Instrument hit rate, miss rate, load duration/failure, eviction, total weight, entry cardinality, and rejected admission. A low hit rate with high retention suggests the cache is storing cost without delivering reuse.

## Async Work Retains More Than the Runnable

A queued task can capture the enclosing object, request body, security context, tracing data, and result promise.

```
executor.submit(() -> process(largeRequest));
```

The lambda may keep `largeRequest` and possibly `this` reachable until the task runs and its future and completion graph are released. Long chains of `CompletableFuture` stages can retain intermediate values, failures, and callbacks. Timeouts do not automatically remove underlying queued or running work.

Inspect queue age and pending-stage counts together with heap paths. The durable fix is bounded admission and cancellation ownership, not nulling arbitrary fields. On shutdown or client cancellation, remove work when safe and ensure completed futures are not retained in diagnostic or retry registries forever.

## ThreadLocal and Virtual-Thread Lifetime

On platform-thread pools, one forgotten value can survive thousands of requests because the worker survives. Always restore or remove context in `finally`, preferably through framework-supported propagation.

Virtual threads usually have a shorter lifetime, reducing cross-request reuse, but `ThreadLocal` values still consume heap for every live virtual thread. Creating a large context or cache in hundreds of thousands of virtual threads can be a memory-capacity problem. Scoped, immutable request data and structured lifetime

remain better than treating thread-local storage as free.

The leak question is always lifetime alignment: does the holder live longer than the value should? Platform workers, global registries, scheduled tasks, and parent class loaders are suspicious because their lifetime is commonly process-wide.

## Finalization, Cleaner Backlogs, and Native Retention

Finalization is deprecated for removal and delays reclamation through an additional lifecycle. If finalizer processing cannot keep up, finalizable objects and their graphs accumulate. Cleaner or phantom-reference cleanup avoids some finalization problems but is still asynchronous and nondeterministic.

For a native resource, the preferred contract is explicit `close()` with try-with-resources. A cleaner can be a last-resort safety net. Its cleanup action must not strongly retain the object it is supposed to clean, and cleanup throughput must be monitored if resources are significant.

Native libraries can retain memory in arenas, thread-local caches, compressed data, TLS structures, or allocator free lists. Memory freed by application code may remain in a native allocator rather than immediately returning to the OS, so RSS can stay high without monotonically growing live allocations. Compare NMT, library counters, native profiles, and RSS over repeated workload/rest cycles.

## Heap-Dump Analysis as a Sequence of Questions

Do not begin by scanning random large classes. Use a repeatable sequence:

1. Confirm dump time, process, heap occupancy, workload phase, and whether the dump includes only live objects.
2. Review largest dominators and retained sets.
3. Compare suspicious class counts with expected business cardinality.
4. Select representative instances and trace paths to strong GC roots.
5. Identify the first owner in application code whose lifetime is too long.
6. Check whether one owner dominates the graph or several independent roots exist.
7. Translate the path into a missing bound, removal, cancellation, or shutdown rule.

An analyzer's "leak suspect" is a hypothesis based on graph shape. A large legitimate product catalog may dominate the heap by design. Domain cardinality and service requirements decide whether retention is valid.

When comparing dumps, normalize for traffic and warm-up. A later dump can be larger merely because caches are warmer or a batch phase differs. Track instance deltas and retained ownership, but verify the same code and operating state. Histograms between full dumps can provide a cheaper time series.

## Live-Only Versus All-Object Evidence

Some diagnostic commands can request a collection or limit output to live objects; other modes include objects that are currently unreachable but not yet reclaimed. Live-only evidence reduces noise for retention analysis but may trigger expensive collection and alter the state being observed. All-object data better represents allocation debris at that instant but can overstate what will remain.

Choose according to the question. For "what owns the post-GC live set?" live objects are appropriate. For an allocation storm, JFR allocation sampling and GC logs may be better than forcing a full collection. Record how the artifact was produced so another engineer does not compare unlike dumps.

## OOME Recovery Is Not an Ordinary Catch Block

After `OutOfMemoryError`, the process may lack memory to allocate error responses, log buffers, recovery tasks, or new threads. Catching OOME deep in business code and continuing can leave partial operations and repeated failures. It may also lose the original evidence through further allocation.

At platform level, decide whether the service should emit minimal diagnostics and terminate for supervised restart. `ExitOnOutOfMemoryError` and `CrashOnOutOfMemoryError` have different evidence and restart consequences. Test the chosen policy in a non-production environment, including dump location, disk capacity, pod termination grace, and alert delivery.

Do not attempt to preallocate a giant emergency buffer as a substitute for ownership design. Small defensive reserves may support a narrow runtime strategy, but recovery correctness still depends on which resource was exhausted and whether application state remains trustworthy.

## A Strong Interview Explanation

Define a Java heap leak as unnecessary reachability from a GC root, then distinguish it from legitimate capacity shortage, allocation bursts, and native exhaustion. Explain that post-GC live-set growth under comparable workload is evidence, not proof. Common retaining owners include unbounded or high-cardinality caches, listeners, thread locals, queues, pending futures, class loaders, and native-resource wrappers.

For diagnosis, preserve the exact OOME detail and environment state. Use GC logs for occupancy dynamics, histograms for class trends, heap dumps for dominators and root paths, JFR for allocation and old-object evidence, NMT for HotSpot native categories, and OS/container data for the total process. Finish with the repair invariant: a bound, lifecycle close, cancellation path, eviction policy, or loader cleanup, plus a metric proving the retained population stabilizes.

## Production Perspective

Do not repeatedly force full GC to decide whether a leak exists in a latency-sensitive service. Use normal post-GC occupancy, controlled diagnostics, and JFR. Do not restart before collecting evidence unless availability risk requires it; a restart destroys the reference graph.

A leak fix should include an invariant and metric: maximum cache weight, queue bound, registration lifecycle, loader count, direct-memory budget, or thread limit. Without a bound, the same failure can return under higher traffic.

## Interview-Ready Summary

A Java leak is an unwanted strong path from a GC root. Common owners are unbounded caches, listeners, ThreadLocals, queues, static registries, class loaders, and native resources. Rising post-GC live set under comparable load is stronger evidence than ordinary sawtooth heap usage.

OutOfMemoryError detail messages identify different memory areas. Container OOM kill can occur without Java OOME because total process memory includes heap and native areas. Diagnose heap retention with dumps, dominators, retained size, and paths to roots; diagnose native growth with NMT, library metrics, and OS evidence.



## Revision Notes

- A Java heap leak is unnecessary reachability from a GC root.
- Rising post-GC live set is stronger evidence than a sawtooth graph.
- Unbounded concurrent collections are still unbounded.
- Listener registration needs explicit deregistration lifecycle.
- Pool-thread ThreadLocal values must be removed.
- Queue backlog retains task arguments and latency.
- Parent-loaded registries can retain child class loaders.
- Heap dumps do not contain native payload bytes.
- Always preserve the complete OutOfMemoryError detail message.
- Java heap space does not automatically prove a leak.
- Metaspace failure concerns class metadata.
- Native-thread failure can happen with free Java heap.
- Container OOM kill may produce no Java exception or heap dump.
- Leave memory headroom beyond -Xmx.
- Heap dumps are large, sensitive, and operationally intrusive.
- Shallow size differs from retained size.
- Dominators reveal ownership of large retained subgraphs.
- Paths to GC roots explain why an object remains reachable.
- Histograms provide counts but not complete ownership.
- NMT must be enabled at startup and does not track every native allocation.
- A durable fix adds lifecycle, capacity, or eviction policy and verifies its trend.
- Separate leak, legitimate live-set capacity, allocation surge, and oversized objects.
- Cache bounds should reflect weight and key cardinality, not only entry count.
- Pending asynchronous work can retain request graphs and completion callbacks.
- Virtual-thread ThreadLocals still consume heap per live virtual thread.
- Cleaner is a safety net; explicit close remains the primary resource lifecycle.
- Native allocator retention can keep RSS high after logical frees.
- Heap-analyzer suspects require domain-cardinality validation.
- Record whether a dump contains live-only or all objects.
- Catching OutOfMemoryError does not make continued operation reliable.